Refinement and Type Difference in SKJS

Abstract

We formalize the internal type representation used by SKJS and study two related type transformations: (1) predicate-driven *refinement* (control-flow narrowing), and (2) the type-difference operation (diff) used during type inference. We state and prove soundness properties for these transformations, while explicitly documenting the intended sources of incompleteness.

1 Definitions Index

Symbol	Defined In
Tags	Definition 1
$(\mathsf{Payload}_k)_{k\in\mathsf{Tags}}, \mathsf{Val}, \mathsf{rtag}$	Definition 2
SType	Definition 3
tag	Definition 4
A0	Lemma 5
A1	Lemma 6
$\mathsf{STypes}, \mathsf{tags}(\cdot)$	Definition 9
$\mathsf{STC}(\cdot,\cdot)$	Definition 45
TInst	Definition 10
TInsts	Definition 11
TParam	Definition 12
Clause	Definition 13
$\mathsf{Clauses}$	Definition 14
TType	Definition 15
refine (predicate-driven refinement)	Definition 18
Predicate-soundness	Definition 20
diff (soundness P1/P2)	Definition 40

2 Runtime Model and Semantic Interface

Definition 1 (Runtime Tags)**.** *Let* Tags *be the set of runtime tags (the coarse runtime kind of a value). In the current implementation, tags are represented as integers, so one may take* $\mathsf{Tags} = \{0, 1, \ldots, 11\}$.

Definition 2 (Runtime Values and Tags)**.** *Assume runtime values carry their tag as part of the representation. We fix a family of payload sets* $(\mathsf{Payload}_k)_{k\in\mathsf{Tags}}$ *and define*

$$\mathsf{Val} \triangleq \{\, (k, p) \mid k \in \mathsf{Tags} \ \wedge \ p \in \mathsf{Payload}_k \,\}.$$

Define the runtime tag function $\mathsf{rtag} : \mathsf{Val} \to \mathsf{Tags}$ *as the first projection:*

$$\mathsf{rtag}((k, p)) = k.$$

Concrete payloads (for this document). *We make primitive payloads concrete and leave structured payloads abstract:*

$\mathsf{Payload}_0 = \{\star\}$ $\quad \mathsf{Payload}_1 = \{\star\}$ $\quad \mathsf{Payload}_2 = \{\star\}$
$\mathsf{Payload}_3 = \{\mathsf{true}, \mathsf{false}\}$ $\quad \mathsf{Payload}_4 = \mathbb{R}$ $\quad \mathsf{Payload}_5 = \mathsf{String}$
$\mathsf{Payload}_6 = \mathbb{Z}$ $\quad \mathsf{Payload}_{10} = \mathsf{String} \times \mathsf{Payload}'_{10}$ *(for some set* $\mathsf{Payload}'_{10}$*).*

For the remaining tags $k \in \{7, 8, 9, 11\}$ *we leave* $\mathsf{Payload}_k$ *as arbitrary sets.*

Definition 3 (Simple Types)**.** *Let* SType *be the set of* simple types*, i.e. the atomic type forms that have a single runtime tag. Formally,* SType *is the smallest set generated by the following constructors:*

- *primitive atoms:* `undefined`*,* `void`*,* `null`*,* `boolean`*,* `number`*,* `string`*,* `bigint`
- *boolean literals:* `true`*,* `false`
- *numeric literals:* n *for* $n \in \mathbb{R}$
- *string literals: "*x*" for strings* x
- *structured atoms:* `tuple`$(\ldots)$*,* `Promise`$(\ldots)$*,* `object`$(\ldots)$*,* `algebraic`$(\ldots)$ *(tagged unions/ADTs with a runtime discriminant),* `function`$(\ldots)$

We write these constructors suggestively; most internal parameters are left abstract. However, for control-flow refinement we will require a concrete semantic interface for object shapes (fields and index signatures); see Definition 7.

Definition 4 (Simple Types and Tags)**.** *The function* $\mathsf{tag} : \mathsf{SType} \to \mathsf{Tags}$ *assigns tags:*

$$\mathsf{tag}(\texttt{undefined}) = 0 \qquad \mathsf{tag}(\texttt{void}) = 1$$
$$\mathsf{tag}(\texttt{null}) = 2 \qquad \mathsf{tag}(\texttt{boolean}), \mathsf{tag}(\texttt{true}), \mathsf{tag}(\texttt{false}) = 3$$
$$\mathsf{tag}(\texttt{number}), \mathsf{tag}(n) = 4 \text{ for } n \in \mathbb{R} \qquad \mathsf{tag}(\texttt{string}), \mathsf{tag}(\text{“}x\text{”}) = 5$$
$$\mathsf{tag}(\texttt{bigint}) = 6 \qquad \mathsf{tag}(\texttt{tuple}(\ldots)) = 7$$
$$\mathsf{tag}(\texttt{Promise}(\ldots)) = 8 \qquad \mathsf{tag}(\texttt{object}(\ldots)) = 9$$
$$\mathsf{tag}(\texttt{algebraic}(\ldots)) = 10 \qquad \mathsf{tag}(\texttt{function}(\ldots)) = 11$$

The semantic interpretation $[\![\cdot]\!] : \mathsf{SType} \to \mathcal{P}(\mathsf{Val})$ *maps each simple type to its set of inhabitants. We define* $[\![\cdot]\!]$ *concretely for primitive types and literals, and define the remaining structured cases*

at tag level (with the `object` *and* `algebraic` *cases further constrained in Definitions 7 and 8):*

$$\begin{aligned}
[\![\texttt{undefined}]\!] &\triangleq \{(0,\star)\} & [\![\texttt{void}]\!] &\triangleq \{(1,\star)\} & [\![\texttt{null}]\!] &\triangleq \{(2,\star)\}\\
[\![\texttt{boolean}]\!] &\triangleq \{(3,b) \mid b \in \{\mathsf{true},\mathsf{false}\}\} & [\![\texttt{true}]\!] &\triangleq \{(3,\mathsf{true})\} & [\![\texttt{false}]\!] &\triangleq \{(3,\mathsf{false})\}\\
[\![\texttt{number}]\!] &\triangleq \{(4,n) \mid n \in \mathbb{R}\} & [\![n]\!] &\triangleq \{(4,n)\} & (n \in \mathbb{R}) &\\
[\![\texttt{string}]\!] &\triangleq \{(5,x) \mid x \in \mathsf{String}\} & [\![\text{“}x\text{”}]\!] &\triangleq \{(5,x)\} & (x \in \mathsf{String}) &\\
[\![\texttt{bigint}]\!] &\triangleq \{(6,z) \mid z \in \mathbb{Z}\} & & & &\\
[\![\texttt{tuple}(\ldots)]\!] &\triangleq \{v \in \mathsf{Val} \mid \mathsf{rtag}(v) = 7\} & & & &\\
[\![\texttt{Promise}(\ldots)]\!] &\triangleq \{v \in \mathsf{Val} \mid \mathsf{rtag}(v) = 8\} & & & &\\
[\![\texttt{function}(\ldots)]\!] &\triangleq \{v \in \mathsf{Val} \mid \mathsf{rtag}(v) = 11\}. & & & &
\end{aligned}$$

Lemma 5 (**A0** (Runtime Tag Consistency))**.** *The semantic interpretation of simple types agrees with runtime tags:*

$$\forall s \in \mathsf{SType}.\ \forall v \in \mathsf{Val}.\ v \in [\![s]\!] \implies \mathsf{rtag}(v) = \mathsf{tag}(s).$$

Proof. By case analysis on s. For primitives and literals, the definition of $[\![s]\!]$ contains only values of the form $(\mathsf{tag}(s), p)$, so $\mathsf{rtag}(v) = \mathsf{tag}(s)$. For the tag-level structured cases (`tuple`, `Promise`, `function`), $[\![s]\!]$ is defined as $\{v \mid \mathsf{rtag}(v) = k\}$ where $k = \mathsf{tag}(s)$. For `object` and `algebraic`, Definitions 7 and 8 explicitly require membership in ObjVal (tag 9) and AlgVal (tag 10) respectively, and these tags agree with tag. □

Lemma 6 (**A1** (Tag Disjointness))**.**

$$\forall s_1, s_2 \in \mathsf{SType}.\ \mathsf{tag}(s_1) \neq \mathsf{tag}(s_2) \implies [\![s_1]\!] \cap [\![s_2]\!] = \emptyset$$

Proof. Let $k_1 = \mathsf{tag}(s_1)$ and $k_2 = \mathsf{tag}(s_2)$ with $k_1 \neq k_2$. By Lemma 5, every $v \in [\![s_1]\!]$ satisfies $\mathsf{rtag}(v) = k_1$ and every $v \in [\![s_2]\!]$ satisfies $\mathsf{rtag}(v) = k_2$. No value can have two different tags, so $[\![s_1]\!] \cap [\![s_2]\!] = \emptyset$. □

Definition 7 (Object Values and Object-Type Semantics)**.** *Let* $\mathsf{ObjVal} \subseteq \mathsf{Val}$ *be the set of object-tagged runtime values:*

$$\mathsf{ObjVal} \triangleq \{v \in \mathsf{Val} \mid \mathsf{rtag}(v) = 9\}.$$

Assume a (partial) finite-property interface on objects:

$$\begin{aligned}
\mathsf{props} &: \mathsf{ObjVal} \to \mathcal{P}_{\mathrm{fin}}(\mathsf{String})\\
\mathsf{get} &: \mathsf{ObjVal} \times \mathsf{String} \rightharpoonup \mathsf{Val}
\end{aligned}$$

such that $\mathsf{get}(o,p)$ *is defined exactly when* $p \in \mathsf{props}(o)$*. Write* $\mathsf{has}(o,p)$ *for* $p \in \mathsf{props}(o)$*.*

Object field specifications. *An* object field specification *is a triple* $F = (F_{\mathrm{req}}, F_{\mathrm{opt}}, F_{\mathrm{idx}})$ *where:*

- $F_{\mathrm{req}} : \mathsf{String} \rightharpoonup \mathsf{TType}$ *is a finite partial map of required fields,*
- $F_{\mathrm{opt}} : \mathsf{String} \rightharpoonup \mathsf{TType}$ *is a finite partial map of optional fields,*
- F_{idx} *is either* None *or* $\mathsf{Some}(K, V)$ *for* $K, V \in \mathsf{TType}$ *(an index signature).*

Semantics of $\texttt{object}(F)$**.** *We assume the object constructor in* SType *can carry such an* F*, and define:*

$$o \in [\![\texttt{object}(F)]\!]$$

if and only if $o \in \mathsf{ObjVal}$ *and the following constraints hold:*

- ***Required fields:*** *for every* $p \in \mathrm{dom}(F_{\mathrm{req}})$*,* $\mathsf{has}(o,p)$ *and* $\mathsf{get}(o,p) \in [\![F_{\mathrm{req}}(p)]\!]$*.*
- ***Optional fields:*** *for every* $p \in \mathrm{dom}(F_{\mathrm{opt}})$*, if* $\mathsf{has}(o,p)$ *then* $\mathsf{get}(o,p) \in [\![F_{\mathrm{opt}}(p)]\!]$ *(and if not, no constraint).*
- ***Index signature (open objects):*** *if* $F_{\mathrm{idx}} = \mathsf{Some}(K,V)$ *then for every* $p \in \mathsf{props}(o)$*: if the runtime string value "*p*" is in* $[\![K]\!]$*, then* $\mathsf{get}(o,p) \in [\![V]\!]$*.*

Note. *This semantics is intentionally* open*: objects may have properties not mentioned in* F_{req} *or* F_{opt}*.*

Definition 8 (Algebraic Values and Tag-Based Algebraic-Type Semantics)**.** *Let* $\mathsf{AlgVal} \subseteq \mathsf{Val}$ *be the set of algebraic-tagged runtime values:*

$$\mathsf{AlgVal} \triangleq \{v \in \mathsf{Val} \mid \mathsf{rtag}(v) = 10\}.$$

Recall that $\mathsf{Payload}_{10} = \mathsf{String} \times \mathsf{Payload}'_{10}$ *(Definition 2). Define the runtime discriminant function* $\mathsf{atag} : \mathsf{AlgVal} \to \mathsf{String}$ *by projection:*

$$\mathsf{atag}((10,(\ell,p))) \triangleq \ell.$$

Semantics of $\texttt{algebraic}(T)$**.** *We assume the algebraic constructor in* SType *can carry a finite set of tags* $T \subseteq_{\mathrm{fin}} \mathsf{String}$*, and define:*

$$v \in [\![\texttt{algebraic}(T)]\!] \quad \textit{iff} \quad v \in \mathsf{AlgVal} \ \wedge \ \mathsf{atag}(v) \in T.$$

Note. *This intentionally models only the discriminant-level behavior needed for refinement; field/constructor payload constraints are left abstract.*

3 Internal Type Representation

Definition 9 (STypes)**.** STypes *is a finite partial map* $\mathsf{Tags} \rightharpoonup \mathsf{SType}$ *with at most one entry per tag: if* $S(k) = s$ *then* $\mathsf{tag}(s) = k$*.*

Partial-map conventions. *We write* $\mathrm{dom}(S)$ *for the domain of defined keys. When a definition returns "undefined" at a key* k*, that means precisely* $k \notin \mathrm{dom}(S)$ *(i.e. the partial map has no entry at* k*).*

$$[\![S]\!] = \bigcup_{k \in \mathrm{dom}(S)} [\![S(k)]\!]$$

We write $\mathsf{tags}(S) = \mathrm{dom}(S)$*. We write* $\{k \mapsto s\}$ *for the singleton partial map with domain* $\{k\}$*.*

Intuition. *An* STypes *value represents the* tag-level *part of a union type: it records, for each runtime tag* k *that may occur, a single representative simple type* s *of that tag. The map shape enforces a normalization invariant ("at most one entry per tag"), which matches the implementation: many operations reason by tags and intentionally do not keep multiple alternatives of the same tag.*

Example. *The union* `number | string` *is represented as:*

$$S = \{4 \mapsto \texttt{number},\ 5 \mapsto \texttt{string}\}$$

and has semantics $[\![S]\!] = [\![\texttt{number}]\!] \cup [\![\texttt{string}]\!]$. *In contrast,* `number` *and the numeric literal* `1` *share the same tag* 4*, so an* STypes *value cannot simultaneously store both; this is why, for example, a tag-based "diff" cannot precisely express* `number \ 1`*.*

Definition 10 (Type Instances)**.** *Let* TInst *be the set of type instances (type variables or generic instantiations).*

Examples. *A type variable instance is something like* T *(a unification variable introduced during type checking; this is* not *a source-level type parameter). A generic instantiation instance is a named type applied to type arguments, e.g.* `Promise⟨number⟩` *or* `Array⟨string⟩`*.*

Each instance $i \in$ TInst *has a semantic interpretation* $[\![i]\!] \subseteq$ Val*.*

Definition 11 (Type Instance Sets)**.** *Let* $\mathsf{TInsts} = \mathcal{P}_{\text{fin}}(\mathsf{TInst})$ *be the set of* finite *sets of instances. For* $I \in$ TInsts*:*

$$[\![I]\!] = \bigcup_{i \in I} [\![i]\!].$$

Definition 12 (Type Parameters)**.** *Let* TParam *be the set of* type parameters *(generic binders), i.e. names introduced by source-level generic declarations such as* `function f⟨T⟩(...)` *or* `type Box⟨T⟩ = ...`*. They differ from type-variable instances: type parameters are binders whose meaning is fixed by the caller/instantiation, whereas type-variable instances are inference unknowns whose meaning is solved by the type checker.*

Definition 13 (Clause)**.** *A clause* $c = (\mathsf{vars}, \mathsf{stypes})$ *consists of:*

- $\mathsf{vars} \subseteq$ TParam*: a finite set of type parameters*
- $\mathsf{stypes} \in \{\mathsf{None}\} \cup \{\mathsf{Some}(S) \mid S \in \mathsf{STypes}\}$

Each type parameter $v \in$ TParam *has an interpretation* $[\![v]\!] \subseteq$ Val*. Semantics:*

$$[\![(\mathsf{vars}, \mathsf{stypes})]\!] = \left(\bigcap_{v \in \mathsf{vars}} [\![v]\!] \right) \cap [\![\mathsf{stypes}]\!]$$

where $[\![\mathsf{None}]\!] = \mathsf{Val}$ *(top) and* $[\![\mathsf{Some}(S)]\!] = [\![S]\!]$*. For* $\mathsf{vars} = \emptyset$*,* $\bigcap_{v \in \emptyset} [\![v]\!] = \mathsf{Val}$*.*

Well-formedness: $\mathsf{stypes} = \mathsf{None} \implies \mathsf{vars} = \emptyset$ *(i.e.,* $(\emptyset, \mathsf{None})$ *is the unique top clause).*

Definition 14 (Clauses)**.** Clauses *is a finite map from* $\mathcal{P}_{\text{fin}}(\mathsf{TParam})$ *to* Clause*, with at most one clause per key, and such that keys agree with clause variables: if* $C(k) = (\mathsf{vars}, \mathsf{stypes})$ *then* $\mathsf{vars} = k$*.*

$$[\![C]\!] = \bigcup_{k \in \mathrm{dom}(C)} [\![C(k)]\!]$$

Example. *Let* $S_{\mathsf{num}} = \{4 \mapsto \texttt{number}\}$ *and* $S_{\mathsf{str}} = \{5 \mapsto \texttt{string}\}$. *Let* $A, B \in \mathsf{TParam}$ *be two type parameters. Consider the clauses map* C *with two entries:*

$$C(\{A\}) = (\{A\}, \mathsf{Some}(S_{\mathsf{num}})) \qquad C(\{B\}) = (\{B\}, \mathsf{Some}(S_{\mathsf{str}})).$$

Then

$$[\![C]\!] = \big([\![A]\!] \cap [\![\texttt{number}]\!]\big) \cup \big([\![B]\!] \cap [\![\texttt{string}]\!]\big).$$

To illustrate a non-singleton key, consider instead the single-entry map C' *defined by:*

$$C'(\{A, B\}) = (\{A, B\}, \mathsf{Some}(S_{\mathsf{num}})).$$

Then $[\![C']\!] = [\![A]\!] \cap [\![B]\!] \cap [\![\texttt{number}]\!]$, *i.e. the clause requires* both *parameters.*

Definition 15 (TType)**.** $\mathsf{TType} = (\mathsf{insts}, \mathsf{clauses})$ *where* $\mathsf{insts} \in \mathsf{TInsts}$ *and* $\mathsf{clauses} \in \mathsf{Clauses}$.

$$[\![(\mathsf{insts}, \mathsf{clauses})]\!] = [\![\mathsf{insts}]\!] \cup [\![\mathsf{clauses}]\!]$$

Schematic. *It is helpful to keep the following picture in mind:*

$$\underbrace{(\mathsf{insts}, \mathsf{clauses})}_{\mathsf{TType}} \quad = \quad \underbrace{\mathsf{insts}}_{\text{“instance” part}} \quad \cup \quad \underbrace{\mathsf{clauses}}_{\text{“conditional” part}}$$

and later $\mathsf{diff}_{\mathsf{TType}}$ *simply applies* diff *componentwise to these two parts.*

Example. *Let* $I = \{\texttt{Promise}\langle\texttt{number}\rangle\} \in \mathsf{TInsts}$ *and let* C *be the clauses map from the previous example. Then* $t = (I, C)$ *denotes the type whose semantics is:*

$$[\![t]\!] = [\![\texttt{Promise}\langle\texttt{number}\rangle]\!] \cup [\![C]\!].$$

4 Subtyping (Skeleton)

We introduce a subtyping judgement on internal types, written $t_1 \leq t_2$ for $t_1, t_2 \in \mathsf{TType}$. In this document, $\leq$ is intended to be a faithful mathematical model of the subtyping *validator* implemented by SKJS (the accept/reject check used by the type checker), rather than an abstract, complete, semantic inclusion relation.

Soundness theorem (to be proved). The central meta-theoretic property we ultimately want from the validator-model judgement $\leq$ is semantic soundness:

$$t_1 \leq t_2 \quad \Longrightarrow \quad [\![t_1]\!] \subseteq [\![t_2]\!].$$

In a full development, this is proved from the subtyping rules/algorithm that define $\leq$.

Incompleteness. The converse implication is intentionally *not* assumed: SKJS may reject some valid semantic inclusions, so $\leq$ can be incomplete.

Note (expansion and substitution effects). In the implementation, the validator operates on expanded/normalized views of types using the current substitution and a variance parameter, and it may update the substitution while checking (as part of inference). We do not model these mechanisms explicitly here; they are absorbed into the meaning of the judgement $t_1 \leq t_2$.

Example obligations (singleton literal types). Once $\leq$ is defined by the validator's rules, it should validate the expected singleton-to-primitive subtypings for primitive literals (these are representative, actually-implemented cases). (In examples we write `1`, `"x"`, `number`, `string` for the corresponding internal types, by abuse of notation.) For example:

$$\texttt{1} \leq \texttt{number} \qquad \texttt{number} \not\leq \texttt{1}.$$

Similarly, a string literal is a subtype of `string`:

$$\texttt{"x"} \leq \texttt{string}.$$

5 Refinement (Predicate-Driven Narrowing)

Definition 16 (Refinement)**.** *Let Γ range over environments mapping local variables to* TType*. Let σ range over runtime environments mapping local variables to runtime values in* Val*. We write $\Gamma(x)$ for lookup and $\Gamma[x \mapsto t]$ for pointwise update.*

Satisfaction. *Define $\sigma \models \Gamma$ to mean: for every local x, we have $\sigma(x) \in [\![\Gamma(x)]\!]$.*

Refinement judgements. *We write:*

$$\Gamma \vdash e \Rightarrow \Gamma' \qquad \textit{and} \qquad \Gamma \vdash e \Leftarrow \Gamma'$$

to mean "refine under the assumption that e evaluates to `true`/`false`" respectively. These judgements are defined by the inference rules in Section 5.1, and are intentionally partial: they only model the refinement patterns implemented in SKJS.

Soundness. *We call the judgement system* branch-sound *if:*

- *whenever $\sigma \models \Gamma$ and e evaluates to `true` under σ (according to the evaluation relation for refinement tests in Definition 17), then $\sigma \models \Gamma'$ holds for every derivation $\Gamma \vdash e \Rightarrow \Gamma'$;*
- *whenever $\sigma \models \Gamma$ and e evaluates to `false` under σ, then $\sigma \models \Gamma'$ holds for every derivation $\Gamma \vdash e \Leftarrow \Gamma'$.*

Implementation correspondence. In SKJS, control-flow narrowing is implemented by rewriting the internal type for a local variable using a tag-indexed filter/map on simple types (see the `FilterType` transformer and `setLocal` in the implementation). This motivates defining refinements compositionally by lifting a transformation on SType values to STypes, Clause, Clauses, and finally TType.

Definition 17 (Evaluation for Refinement Tests)**.** *We define a partial evaluation relation for the refinement-test fragment:*

$$\sigma \models_{\mathsf{true}} e \qquad \textit{and} \qquad \sigma \models_{\mathsf{false}} e.$$

It is defined only for the supported test forms:

- ***Nullish:*** *$\sigma \models_{\mathsf{true}} (x \texttt{ === } \texttt{null})$ iff $\mathsf{rtag}(\sigma(x)) = 2$; $\sigma \models_{\mathsf{false}}$ is the negation. Similarly for `undefined`, with $\mathsf{rtag}(\sigma(x)) \in \{0, 1\}$.*

- ***typeof:*** $\sigma \models_{\mathsf{true}} (\texttt{typeof}\ x === k)$ *iff* $\sigma(x) \in P_{\mathsf{typeof}=k}$ *where* $P_{\mathsf{typeof}=k}$ *is as in Definition 32; the false relation is negation.*
- ***in:*** $\sigma \models_{\mathsf{true}} (\texttt{"}p\texttt{"}\ \texttt{in}\ x)$ *iff* $\sigma(x) \in \mathsf{ObjVal}$ *and* $\mathsf{has}(\sigma(x), p)$*; the false relation is negation.*
- ***Algebraic tag equality:*** $\sigma \models_{\mathsf{true}} (x._\texttt{tag} === \ell)$ *iff* $\sigma(x) \in \mathsf{AlgVal}$ *and* $\mathsf{atag}(\sigma(x)) = \ell$*, where* $\ell \in \mathsf{String}$ *is a string literal; the false relation is negation.*
- ***instanceof:*** $\sigma \models_{\mathsf{true}} (x\ \texttt{instanceof}\ C)$ *iff* $\sigma(x) \in P_{\mathsf{inst}}(C)$ *for a predicate* $P_{\mathsf{inst}}(C) \subseteq \mathsf{Val}$ *(see Definition 38); the false relation is negation.*
- ***Connectives:*** $\sigma \models_{\mathsf{true}} (e_1\ \&\&\ e_2)$ *iff* $\sigma \models_{\mathsf{true}} e_1$ *and* $\sigma \models_{\mathsf{true}} e_2$*.* $\sigma \models_{\mathsf{false}} (e_1\ ||\ e_2)$ *iff* $\sigma \models_{\mathsf{false}} e_1$ *and* $\sigma \models_{\mathsf{false}} e_2$*. (Other combinations are omitted, since SKJS only uses these connective cases for refinement in the current implementation.)*

Definition 18 (Filter/Map Refinement (Implementation Model))**.** *Let* $P \in \mathcal{P}(\mathsf{Val})$ *be a predicate. Let* $f : \mathsf{SType} \to \{\mathsf{true}, \mathsf{false}\}$ *be a filter and* $g : \mathsf{SType} \to \mathsf{SType}$ *be a mapper. We say* (f, g) *is* sound for P *if for every* $s \in \mathsf{SType}$ *and every* $v \in [\![s]\!]$*:*

$$v \in P \implies (f(s) = \mathsf{true}\ \wedge\ v \in [\![g(s)]\!]).$$

Remark (tag preservation). *In the implementation, the mapper* g *used for refinement preserves runtime tags:* $\mathsf{tag}(g(s)) = \mathsf{tag}(s)$*. We do not require this for the semantic statement below, but it ensures the lifted* STypes *transformation does not merge distinct tags.*

Lift to STypes**.** *Define:*

$$\mathsf{refine}^{f,g}_{\mathsf{STypes}}(S) \triangleq \{\, \mathsf{tag}(g(S(k))) \mapsto g(S(k)) \mid k \in \mathrm{dom}(S),\ f(S(k)) = \mathsf{true} \,\}.$$

Operationally, this is exactly the "filter-map" over the unique representative per tag.

Lift to Clause**.** *For a clause* $c = (\mathsf{vars}, \mathsf{stypes})$ *define:*

$$\mathsf{refine}^{f,g}_{\mathsf{Clause}}(c) \triangleq \begin{cases} (\mathsf{vars}, \mathsf{None}) & \text{if } \mathsf{stypes} = \mathsf{None} \\ (\mathsf{vars}, \mathsf{Some}(\mathsf{refine}^{f,g}_{\mathsf{STypes}}(S))) & \text{if } \mathsf{stypes} = \mathsf{Some}(S) \end{cases}$$

and interpret the special case $\mathsf{Some}(\emptyset)$ *as* $\bot$ *(dropped), matching the implementation invariant that bottom clauses are not stored.*

Lift to Clauses**.** *For* $C \in \mathsf{Clauses}$*, define* $\mathsf{refine}^{f,g}_{\mathsf{Clauses}}(C)$ *by applying* $\mathsf{refine}^{f,g}_{\mathsf{Clause}}$ *to every clause and dropping* $\bot$ *results.*

Lift to TType**.** *For* $t = (\mathsf{insts}, \mathsf{clauses})$ *define:*

$$\mathsf{refine}^{f,g}(t) \triangleq (\mathsf{insts}, \mathsf{refine}^{f,g}_{\mathsf{Clauses}}(\mathsf{clauses})).$$

This matches the current implementation: refinement rewrites the clause part but does not filter the instance part.

Definition 19 (Defaulting Refinement (Model of `setLocal`))**.** *Some refinements in SKJS provide a* default *type when the current type is too imprecise to refine (notably when it is top or has no informative alternatives). This is modeled by a* defaulting refinement *operator.*

Let $R : \mathsf{TType} \to \mathsf{TType}$ *be a transformer and let* d *be an* optional *default simple type (*$d \in \{\mathsf{None}\} \cup \{\mathsf{Some}(s) \mid s \in \mathsf{SType}\}$*).*

Singleton constructor. *Write* $\mathsf{single}(s)$ *for the* TType *value consisting of a single clause with* $\mathsf{STypes} = \{\mathsf{tag}(s) \mapsto s\}$ *and no instances.*

Top/empty test. *We say a* TType *value is* syntactically top *if its clause component is the unique top clause* $(\emptyset, \mathsf{None})$*, and* syntactically empty *if its clause component is empty (bottom).*

Definition. *Define* $\mathsf{def}(R, d) : \mathsf{TType} \to \mathsf{TType}$ *by:*

$$\mathsf{def}(R, d)(t) \triangleq \begin{cases} R(t) & \text{if } d = \mathsf{None} \\ \mathsf{single}(s) & \text{if } d = \mathsf{Some}(s) \text{ and } R(t) \text{ is syntactically top or empty} \\ R(t) & \text{otherwise} \end{cases}$$

Note. *This abstracts the implementation check in* `setLocal` *based on* `getSTypes` *(after expansion): it injects a default when refinement would otherwise leave a top/empty shape.*

Definition 20 (Predicate-Sound Type Transformer)**.** *Let* $P \in \mathcal{P}(\mathsf{Val})$ *and let* $R : \mathsf{TType} \to \mathsf{TType}$*. We say* R *is* predicate-sound for P *if for all* $t \in \mathsf{TType}$*:*

$$[\![t]\!] \cap P \subseteq [\![R(t)]\!].$$

Lemma 21 (Filter/Map Refinement is Predicate-Sound)**.** *If* (f, g) *is sound for* P*, then the transformer* $t \mapsto \mathsf{refine}^{f,g}(t)$ *is predicate-sound for* P *(Definition 20).*

Proof. Let $t = (\mathsf{insts}, \mathsf{clauses})$. We prove $[\![t]\!] \cap P \subseteq [\![\mathsf{refine}^{f,g}(t)]\!]$ by cases on membership in $[\![\mathsf{insts}]\!] \cup [\![\mathsf{clauses}]\!]$.

- If $v \in [\![\mathsf{insts}]\!] \cap P$, then $v \in [\![\mathsf{insts}]\!] \subseteq [\![\mathsf{refine}^{f,g}(t)]\!]$ since refinement leaves insts unchanged.
- If $v \in [\![\mathsf{clauses}]\!] \cap P$, then $v \in [\![(\mathsf{vars}, \mathsf{stypes})]\!]$ for some clause $(\mathsf{vars}, \mathsf{stypes})$ in clauses. By clause semantics (Definition 13), $v \in \bigcap_{w \in \mathsf{vars}} [\![w]\!]$ and $v \in [\![\mathsf{stypes}]\!]$. If $\mathsf{stypes} = \mathsf{None}$, then $\mathsf{refine}^{f,g}_{\mathsf{Clause}}(\mathsf{vars}, \mathsf{None}) = (\mathsf{vars}, \mathsf{None})$ and the clause remains top, so v is still included. If $\mathsf{stypes} = \mathsf{Some}(S)$, then $v \in [\![S]\!] = \bigcup_{k \in \mathrm{dom}(S)} [\![S(k)]\!]$, so $v \in [\![S(k)]\!]$ for some $k \in \mathrm{dom}(S)$. By soundness of (f, g) for P and $v \in P$, we have $f(S(k)) = \mathsf{true}$ and $v \in [\![g(S(k))]\!]$. Hence $v \in [\![\mathsf{refine}^{f,g}_{\mathsf{STypes}}(S)]\!]$. The refined clause is $(\mathsf{vars}, \mathsf{Some}(\mathsf{refine}^{f,g}_{\mathsf{STypes}}(S)))$, and since vars is unchanged and $v \in \bigcap_{w \in \mathsf{vars}} [\![w]\!]$, we have $v \in [\![(\mathsf{vars}, \mathsf{Some}(\mathsf{refine}^{f,g}_{\mathsf{STypes}}(S)))]\!]$. Therefore $v \in [\![\mathsf{refine}^{f,g}_{\mathsf{Clauses}}(\mathsf{clauses})]\!]$.

Combining cases yields the desired inclusion. □

Lemma 22 (Defaulting Preserves Predicate-Soundness)**.** *Let* $R : \mathsf{TType} \to \mathsf{TType}$ *be predicate-sound for* $P \subseteq \mathsf{Val}$*. Let* d *be an optional default. If* $d = \mathsf{None}$*, then* $\mathsf{def}(R, d)$ *is predicate-sound for* P*. If* $d = \mathsf{Some}(s)$ *and* $P \subseteq [\![\mathsf{single}(s)]\!]$*, then* $\mathsf{def}(R, d)$ *is predicate-sound for* P*.*

Proof. If $d = \mathsf{None}$, then $\mathsf{def}(R, d) = R$ by Definition 19, so predicate-soundness is immediate. If $d = \mathsf{Some}(s)$, let $t \in \mathsf{TType}$ and let $v \in [\![t]\!] \cap P$. If $R(t)$ is not syntactically top/empty, then $\mathsf{def}(R, d)(t) = R(t)$ and $v \in [\![R(t)]\!]$ by predicate-soundness of R. If $R(t)$ is syntactically top/empty, then $\mathsf{def}(R, d)(t) = \mathsf{single}(s)$, and by hypothesis $v \in P \subseteq [\![\mathsf{single}(s)]\!]$. Thus $[\![t]\!] \cap P \subseteq [\![\mathsf{def}(R, d)(t)]\!]$ for all t. □

5.1 Algorithmic Refinement Judgements (Inference-Rule Style)

Local update helper. Given a local x and a refinement transformer $R : \mathsf{TType} \to \mathsf{TType}$, define:

$$\mathsf{upd}(\Gamma, x, R) \triangleq \Gamma[x \mapsto R(\Gamma(x))].$$

When a rule refines a non-local expression, it produces Γ.

Default behavior. If no rule applies to e (i.e. e is not one of the supported refinement-test forms), then refinement leaves the environment unchanged.

Strict inequality. The implementation treats strict inequality `!==` by flipping the target (i.e. refining for `false` of the corresponding strict equality). Accordingly, the refinement rules for $(x \mathbin{!==} y)$ are the same as for $(x === y)$, with $\Rightarrow$ and $\Leftarrow$ swapped.

Nullish equality.

$$\frac{}{\Gamma \vdash (x === \texttt{null}) \Rightarrow \mathsf{upd}(\Gamma, x,\, t \mapsto \mathsf{refine}^{f_{\mathsf{null}}, g_{\mathsf{null}}}(t))}\ \text{R-NullT}$$

$$\frac{}{\Gamma \vdash (x === \texttt{null}) \Leftarrow \mathsf{upd}(\Gamma, x,\, t \mapsto \mathsf{refine}^{f_{\neg\mathsf{null}}, g_{\mathsf{null}}}(t))}\ \text{R-NullF}$$

$$\frac{}{\Gamma \vdash (x === \texttt{undefined}) \Rightarrow \mathsf{upd}(\Gamma, x,\, t \mapsto \mathsf{refine}^{f_{\mathsf{undef}}, g_{\mathsf{undef}}}(t))}\ \text{R-UndefT}$$

$$\frac{}{\Gamma \vdash (x === \texttt{undefined}) \Leftarrow \mathsf{upd}(\Gamma, x,\, t \mapsto \mathsf{refine}^{f_{\neg\mathsf{undef}}, g_{\mathsf{undef}}}(t))}\ \text{R-UndefF}$$

Side condition (locals only). In the implementation, comparisons against `null`/`undefined` are only permitted when the compared expression is a local variable; otherwise a typing error is raised. Accordingly, the rules above are intended only for tests where x is a local.

`typeof`.

$$\frac{}{\Gamma \vdash (\texttt{typeof}\ x === k) \Rightarrow \mathsf{upd}(\Gamma, x,\, t \mapsto \mathsf{def}(t' \mapsto \mathsf{refine}^{f_k, g_k}(t'),\, d_k)(t))}\ \text{R-TypeofT}$$

$$\frac{}{\Gamma \vdash (\texttt{typeof}\ x === k) \Leftarrow \mathsf{upd}(\Gamma, x,\, t \mapsto \mathsf{refine}^{\neg f_k, g_k}(t))}\ \text{R-TypeofF}$$

where f_k, g_k, d_k are as in Definition 33.

`typeof "symbol"`. The implementation performs no refinement for `typeof x === "symbol"`; both branches leave the environment unchanged.

`in`.

$$\frac{}{\Gamma \vdash (\texttt{"}p\texttt{"}\ \texttt{in}\ x) \Rightarrow \mathsf{upd}(\Gamma, x,\, t \mapsto \mathsf{def}(t' \mapsto \mathsf{refine}^{f_p^{\mathsf{true}}, g_p^{\mathsf{true}}}(t'),\, d_p^{\mathsf{in}})(t))}\ \text{R-InT}$$

$$\frac{}{\Gamma \vdash (\texttt{"}p\texttt{"}\ \texttt{in}\ x) \Leftarrow \mathsf{upd}(\Gamma, x,\, t \mapsto \mathsf{refine}^{f_p^{\mathsf{false}}, g_p^{\mathsf{false}}}(t))}\ \text{R-InF}$$

where $d_p^{\mathsf{in}} = \mathsf{Some}(\texttt{object}(F_p))$ is a default object type requiring property p with top value type (Definition 7). This corresponds to the implementation's behavior when the left-hand side is a single string literal.

Algebraic tag equality.

$$\frac{}{\Gamma \vdash (x.\texttt{_tag} === \ell) \Rightarrow \mathsf{upd}(\Gamma, x,\, t \mapsto \mathsf{refine}^{f_\ell^{\mathsf{alg}}, g_\ell^{\mathsf{alg,true}}}(t))}\ \text{R-AlgTagT}$$

$$\frac{}{\Gamma \vdash (x.\texttt{_tag} === \ell) \Leftarrow \mathsf{upd}(\Gamma, x,\, t \mapsto \mathsf{refine}^{f_\ell^{\mathsf{alg}}, g_\ell^{\mathsf{alg,false}}}(t))}\ \text{R-AlgTagF}$$

where $\ell \in \mathsf{String}$ is a string literal.

`instanceof`**.**

$$\frac{}{\Gamma \vdash (x\ \texttt{instanceof}\ C) \Rightarrow \mathsf{upd}(\Gamma, x,\, t \mapsto T_{\mathsf{inst}}(C))}\ \text{R-InstT}$$

$$\frac{}{\Gamma \vdash (x\ \texttt{instanceof}\ C) \Leftarrow \Gamma}\ \text{R-InstF}$$

where $T_{\mathsf{inst}}(C)$ is the instance type produced by the implementation (Definition 38).

$$\frac{}{\Gamma \vdash (x\ \texttt{instanceof Array}) \Rightarrow \mathsf{upd}(\Gamma, x,\, t \mapsto T_{\mathsf{array}})}\ \text{R-InstArrayT}$$

where T_{array} is the implementation's chosen array instance type (e.g. $\texttt{Array}\langle\top\rangle$).

Boolean connectives. The implementation composes refinements sequentially. For example:

$$\frac{\Gamma \vdash e_1 \Rightarrow \Gamma_1 \qquad \Gamma_1 \vdash e_2 \Rightarrow \Gamma_2}{\Gamma \vdash (e_1\ \&\&\ e_2) \Rightarrow \Gamma_2}\ \text{R-AndT} \qquad \frac{\Gamma \vdash e_1 \Leftarrow \Gamma_1 \qquad \Gamma_1 \vdash e_2 \Leftarrow \Gamma_2}{\Gamma \vdash (e_1\ ||\ e_2) \Leftarrow \Gamma_2}\ \text{R-OrF}$$

Lemma 23 (Predicate-Sound Transformers Yield Branch-Sound Rules)**.** *Let $R : \mathsf{TType} \to \mathsf{TType}$ be predicate-sound for predicate $P \subseteq \mathsf{Val}$. Suppose the inference rule for expression e updates the environment via $\mathsf{upd}(\Gamma, x, R)$. If $\sigma \models_{\mathsf{true}} e$ implies $\sigma(x) \in P$, then the rule is branch-sound for the true branch. Analogously for the false branch with $\sigma \models_{\mathsf{false}} e$.*

Proof. Suppose $\sigma \models \Gamma$ and $\sigma \models_{\mathsf{true}} e$. Let $\Gamma' = \mathsf{upd}(\Gamma, x, R)$. For any local $y \neq x$, $\Gamma'(y) = \Gamma(y)$ and $\sigma(y) \in [\![\Gamma(y)]\!] = [\![\Gamma'(y)]\!]$. For x: by hypothesis $\sigma(x) \in P$, and by $\sigma \models \Gamma$ we have $\sigma(x) \in [\![\Gamma(x)]\!]$. Thus $\sigma(x) \in [\![\Gamma(x)]\!] \cap P \subseteq [\![R(\Gamma(x))]\!] = [\![\Gamma'(x)]\!]$ by predicate-soundness of R. Hence $\sigma \models \Gamma'$. □

Lemma 24 (Connective Rules are Branch-Sound)**.** *Assume that for every derivation $\Gamma \vdash e_1 \Rightarrow \Gamma_1$, the rule is branch-sound for the true branch in the sense of Definition 16. Assume likewise that for every derivation $\Gamma_1 \vdash e_2 \Rightarrow \Gamma_2$, the rule is branch-sound for the true branch. Then the connective rule **R-AndT** is branch-sound for the true branch.*

*Dually, assume that for every derivation $\Gamma \vdash e_1 \Leftarrow \Gamma_1$ and $\Gamma_1 \vdash e_2 \Leftarrow \Gamma_2$, the rules are branch-sound for the false branch. Then the connective rule **R-OrF** is branch-sound for the false branch.*

Proof. **R-AndT.** Suppose $\sigma \models \Gamma$ and $\sigma \models_{\mathsf{true}} (e_1 \;\&\&\; e_2)$. By Definition 17, we have $\sigma \models_{\mathsf{true}} e_1$ and $\sigma \models_{\mathsf{true}} e_2$. Consider a derivation of $\Gamma \vdash (e_1 \;\&\&\; e_2) \Rightarrow \Gamma_2$ via **R-AndT**; it contains premises $\Gamma \vdash e_1 \Rightarrow \Gamma_1$ and $\Gamma_1 \vdash e_2 \Rightarrow \Gamma_2$. By branch-soundness of the first premise and $\sigma \models_{\mathsf{true}} e_1$, from $\sigma \models \Gamma$ we obtain $\sigma \models \Gamma_1$. Then by branch-soundness of the second premise and $\sigma \models_{\mathsf{true}} e_2$, we obtain $\sigma \models \Gamma_2$. This is exactly branch-soundness of **R-AndT**.

R-OrF. Suppose $\sigma \models \Gamma$ and $\sigma \models_{\mathsf{false}} (e_1 \;||\; e_2)$. By Definition 17, we have $\sigma \models_{\mathsf{false}} e_1$ and $\sigma \models_{\mathsf{false}} e_2$. For a derivation of $\Gamma \vdash (e_1 \;||\; e_2) \Leftarrow \Gamma_2$ via **R-OrF**, we similarly obtain $\sigma \models \Gamma_1$ from the first premise and then $\sigma \models \Gamma_2$ from the second premise. □

Soundness connection. Each of the case-study lemmas in the following subsections establishes that the filter/map pair (f, g) is sound for the corresponding runtime predicate P, which by Lemma 21 implies predicate-soundness of $\mathsf{refine}^{f,g}$, and by Lemma 23 implies branch-soundness of the inference rule.

5.2 Refinement Case Study: the `in` Operator

We formalize the refinement performed for conditions of the form `"p" in x` in SKJS. The implementation refines only local variables and updates object types by either (a) adding a required field on the true branch, or (b) forgetting a field on the false branch.

Definition 25 (`in` Predicates)**.** *Fix a property name $p \in \mathsf{String}$. Define:*

$$P_{\text{in}}^{\mathsf{true}}(p) \triangleq \{\, o \in \mathsf{Val} \mid o \in \mathsf{ObjVal} \wedge \mathsf{has}(o, p) \,\} \qquad P_{\text{in}}^{\mathsf{false}}(p) \triangleq \mathsf{Val} \setminus P_{\text{in}}^{\mathsf{true}}(p).$$

Definition 26 (`in` Filter/Map Pair)**.** *Fix $p \in \mathsf{String}$ and let $\top_{\mathsf{TType}}$ be any type with $[\![\top_{\mathsf{TType}}]\!] = \mathsf{Val}$ (e.g. the unique top clause). Let F_p be the object field specification with F_{req} mapping $p \mapsto \top_{\mathsf{TType}}$, F_{opt} empty, and $F_{\text{idx}} = \mathsf{None}$. Define a mapper $g_p^{\mathsf{true}} : \mathsf{SType} \to \mathsf{SType}$ as follows:*

- *If s is an object type $s = \texttt{object}(F)$ and $F_{\text{idx}} = \mathsf{None}$ and $p \notin \mathrm{dom}(F_{\text{req}})$, then $g_p^{\mathsf{true}}(s)$ is $\texttt{object}(F')$ where F' is F except that F'_{req} additionally maps $p \mapsto \top_{\mathsf{TType}}$.*
- *Otherwise, $g_p^{\mathsf{true}}(s) = s$.*

Let $f_p^{\mathsf{true}} : \mathsf{SType} \to \{\mathsf{true}, \mathsf{false}\}$ be the filter defined by:

$$f_p^{\mathsf{true}}(s) = \begin{cases} \mathsf{true} & \text{if } \mathsf{tag}(s) = 9 \text{ (i.e., } s \text{ is an object type)} \\ \mathsf{false} & \text{otherwise (primitives, tuples, functions, etc.)} \end{cases}$$

This matches the implementation's conservative approximation: only object-tagged types can satisfy the `in` predicate.

Define a mapper $g_p^{\mathsf{false}} : \mathsf{SType} \to \mathsf{SType}$ by:

- *If $s = \texttt{object}(F)$ then $g_p^{\mathsf{false}}(s) = \texttt{object}(F \setminus p)$ where $F \setminus p$ removes p from both F_{req} and F_{opt} (and leaves the index signature unchanged).*
- *Otherwise $g_p^{\mathsf{false}}(s) = s$.*

Let $f_p^{\mathsf{false}}(s) = \mathsf{true}$ for all s (the implementation does not attempt to exclude non-objects in the false branch).

Default. *Define* $d_p^{\mathsf{in}} \triangleq \mathsf{Some}(\texttt{object}(F_p))$.

Lemma 27 (`in`-True Branch (with Default) is Predicate-Sound)**.** *For every* $p \in \mathsf{String}$*, the transformer* $t \mapsto \mathsf{def}(t' \mapsto \mathsf{refine}^{f_p^{\mathsf{true}}, g_p^{\mathsf{true}}}(t'), d_p^{\mathsf{in}})(t)$ *is predicate-sound for* $P_{\mathrm{in}}^{\mathsf{true}}(p)$.

Proof. Let $R_p(t) \triangleq \mathsf{refine}^{f_p^{\mathsf{true}}, g_p^{\mathsf{true}}}(t)$. We first show that $(f_p^{\mathsf{true}}, g_p^{\mathsf{true}})$ is sound for $P_{\mathrm{in}}^{\mathsf{true}}(p)$ (Definition 18), and conclude that R_p is predicate-sound for $P_{\mathrm{in}}^{\mathsf{true}}(p)$ by Lemma 21. Let $s \in \mathsf{SType}$ and $v \in [\![s]\!]$ with $v \in P_{\mathrm{in}}^{\mathsf{true}}(p)$. Then $v \in \mathsf{ObjVal}$, so $\mathsf{rtag}(v) = 9$. By A0, $\mathsf{rtag}(v) = \mathsf{tag}(s)$, hence $\mathsf{tag}(s) = 9$ and therefore s must be an object type of the form $\texttt{object}(F)$. By definition of $P_{\mathrm{in}}^{\mathsf{true}}(p)$, $\mathsf{has}(v, p)$. The filter keeps object types, so $f_p^{\mathsf{true}}(s) = \mathsf{true}$. If g_p^{true} adds the required field $p \mapsto \top_{\mathsf{TType}}$, then v satisfies the added requirement because it already has p and $\mathsf{get}(v, p) \in [\![\top_{\mathsf{TType}}]\!] = \mathsf{Val}$. Otherwise $g_p^{\mathsf{true}}(s) = s$ and membership is immediate. Thus $v \in [\![g_p^{\mathsf{true}}(s)]\!]$ and soundness holds.

Finally, by Definition 7, every $v \in P_{\mathrm{in}}^{\mathsf{true}}(p)$ is in $[\![\texttt{object}(F_p)]\!] = [\![\mathsf{single}(\texttt{object}(F_p))]\!]$, so $P_{\mathrm{in}}^{\mathsf{true}}(p) \subseteq [\![\mathsf{single}(\texttt{object}(F_p))]\!]$. By Lemma 22, $\mathsf{def}(R_p, d_p^{\mathsf{in}})$ is predicate-sound for $P_{\mathrm{in}}^{\mathsf{true}}(p)$. □

Lemma 28 (`in`-False Branch is Predicate-Sound)**.** *For every* $p \in \mathsf{String}$*, the refinement* $\mathsf{refine}^{f_p^{\mathsf{false}}, g_p^{\mathsf{false}}}$ *is predicate-sound for* $P_{\mathrm{in}}^{\mathsf{false}}(p)$.

Proof. We show $(f_p^{\mathsf{false}}, g_p^{\mathsf{false}})$ is sound for $P_{\mathrm{in}}^{\mathsf{false}}(p)$ (Definition 18), and conclude by Lemma 21. Let $s \in \mathsf{SType}$ and $v \in [\![s]\!]$ with $v \in P_{\mathrm{in}}^{\mathsf{false}}(p)$. We have $f_p^{\mathsf{false}}(s) = \mathsf{true}$ by definition. If s is not an object type, then $g_p^{\mathsf{false}}(s) = s$ so $v \in [\![g_p^{\mathsf{false}}(s)]\!]$. If $s = \texttt{object}(F)$, then $g_p^{\mathsf{false}}(s) = \texttt{object}(F \setminus p)$ which removes p from field constraints. By Definition 7, the semantics only imposes constraints on listed fields, so removing a field from the specification can only enlarge (or preserve) the set of inhabitants: $[\![\texttt{object}(F)]\!] \subseteq [\![\texttt{object}(F \setminus p)]\!]$. Hence $v \in [\![g_p^{\mathsf{false}}(s)]\!]$. □

5.3 Refinement Case Study: Nullish Checks

We model the refinements performed for comparisons of a local variable against `null` and `undefined`. In the implementation, `undefined` and `void` are treated equivalently for refinement.

Definition 29 (Nullish Predicates)**.** *Define the following predicates on runtime values:*

$$P_{\mathsf{null}} \triangleq \{v \in \mathsf{Val} \mid \mathsf{rtag}(v) = 2\} \qquad P_{\mathsf{undef}} \triangleq \{v \in \mathsf{Val} \mid \mathsf{rtag}(v) \in \{0, 1\}\}.$$

We also write $\neg P$ *for* $\mathsf{Val} \setminus P$.

Lemma 30 (Null Refinement is Predicate-Sound)**.** *Let* $f_{\mathsf{null}}(s) \triangleq (\mathsf{tag}(s) = 2)$ *and let* $g_{\mathsf{null}}(s) \triangleq s$*. Then* $\mathsf{refine}^{f_{\mathsf{null}}, g_{\mathsf{null}}}$ *is predicate-sound for* P_{null}*. Moreover, letting* $f_{\neg\mathsf{null}}(s) \triangleq (\mathsf{tag}(s) \neq 2)$*,* $\mathsf{refine}^{f_{\neg\mathsf{null}}, g_{\mathsf{null}}}$ *is predicate-sound for* $\neg P_{\mathsf{null}}$.

Proof. We show $(f_{\mathsf{null}}, g_{\mathsf{null}})$ is sound for P_{null} (Definition 18) and apply Lemma 21. Let $s \in \mathsf{SType}$ and $v \in [\![s]\!]$ with $v \in P_{\mathsf{null}}$. Then $\mathsf{rtag}(v) = 2$, and by A0 we have $\mathsf{tag}(s) = 2$, hence $f_{\mathsf{null}}(s) = \mathsf{true}$. Since g_{null} is the identity, $v \in [\![g_{\mathsf{null}}(s)]\!]$ is immediate. The $\neg P_{\mathsf{null}}$ case is analogous: if $v \in \neg P_{\mathsf{null}}$ then $\mathsf{rtag}(v) \neq 2$, so by A0 $\mathsf{tag}(s) \neq 2$ and $f_{\neg\mathsf{null}}(s) = \mathsf{true}$. □

Lemma 31 (Undefined/Void Refinement is Predicate-Sound)**.** *Let* $f_{\mathsf{undef}}(s) \triangleq (\mathsf{tag}(s) \in \{0, 1\})$ *and* $g_{\mathsf{undef}}(s) \triangleq s$*. Then* $\mathsf{refine}^{f_{\mathsf{undef}}, g_{\mathsf{undef}}}$ *is predicate-sound for* P_{undef}*. Moreover, letting* $f_{\neg\mathsf{undef}}(s) \triangleq (\mathsf{tag}(s) \notin \{0, 1\})$*,* $\mathsf{refine}^{f_{\neg\mathsf{undef}}, g_{\mathsf{undef}}}$ *is predicate-sound for* $\neg P_{\mathsf{undef}}$.

Proof. We show $(f_{\mathsf{undef}}, g_{\mathsf{undef}})$ is sound for P_{undef} (Definition 18) and apply Lemma 21. Let $s \in \mathsf{SType}$ and $v \in [\![s]\!]$ with $v \in P_{\mathsf{undef}}$, i.e., $\mathsf{rtag}(v) \in \{0, 1\}$. By A0, $\mathsf{tag}(s) = \mathsf{rtag}(v) \in \{0, 1\}$, so $f_{\mathsf{undef}}(s) = \mathsf{true}$ and $v \in [\![g_{\mathsf{undef}}(s)]\!]$. The $\neg P_{\mathsf{undef}}$ case is analogous. □

5.4 Refinement Case Study: `typeof` Checks

We model the refinements performed for tests of the form `typeof x === "k"` for a subset of strings k. The implementation refines based on the internal tag/classification of the current SType alternatives.

Note. The predicate modeled here is the one used by the checker for narrowing; it may be a conservative approximation of JavaScript runtime `typeof`.

Definition 32 (`typeof` Predicates (Tag-Level))**.** *Define the following tag-level predicates:*

$$\begin{aligned}
P_{\mathsf{typeof}=\texttt{"string"}} &\triangleq \{v \in \mathsf{Val} \mid \mathsf{rtag}(v) = 5\} \\
P_{\mathsf{typeof}=\texttt{"number"}} &\triangleq \{v \in \mathsf{Val} \mid \mathsf{rtag}(v) = 4\} \\
P_{\mathsf{typeof}=\texttt{"bigint"}} &\triangleq \{v \in \mathsf{Val} \mid \mathsf{rtag}(v) = 6\} \\
P_{\mathsf{typeof}=\texttt{"boolean"}} &\triangleq \{v \in \mathsf{Val} \mid \mathsf{rtag}(v) = 3\} \\
P_{\mathsf{typeof}=\texttt{"function"}} &\triangleq \{v \in \mathsf{Val} \mid \mathsf{rtag}(v) = 11\} \\
P_{\mathsf{typeof}=\texttt{"undefined"}} &\triangleq \{v \in \mathsf{Val} \mid \mathsf{rtag}(v) \in \{0, 1\}\} \\
P_{\mathsf{typeof}=\texttt{"object"}} &\triangleq \mathsf{ObjVal}.
\end{aligned}$$

Note. *This definition models the predicate used for refinement soundness; the implementation's filter for `"object"` may conservatively retain other same-tag alternatives. The `"symbol"` case is intentionally omitted because SKJS performs no refinement for it.*

Definition 33 (`typeof` Filters and Defaults)**.** *For each k in Definition 32, we define a filter/map pair and an optional default d_k matching the implementation.*

- *For $k \in \{$`"string"`, `"number"`, `"bigint"`, `"boolean"`, `"function"`, `"undefined"`$\}$, let $f_k(s)$ be true iff $\mathsf{tag}(s)$ is the corresponding tag (or tag-set), and let g_k be the identity.*
- *For $k =$ `"object"`, let $f_{\texttt{"object"}}(s)$ be true iff $\mathsf{tag}(s) \in \{2, 9, 10\}$ (object, null, or algebraic alternatives retained), and let $g_{\texttt{"object"}}$ be the identity.*

Define defaults:

- $d_{\texttt{"string"}} = \mathsf{Some}(\texttt{string})$, $d_{\texttt{"number"}} = \mathsf{Some}(\texttt{number})$, $d_{\texttt{"bigint"}} = \mathsf{Some}(\texttt{bigint})$, $d_{\texttt{"boolean"}} = \mathsf{Some}(\texttt{boolean})$.
- *Let $F_\emptyset$ be the empty object field specification (no required/optional fields, no index signature), and set $d_{\texttt{"object"}} = \mathsf{Some}(\texttt{object}(F_\emptyset))$.*
- $d_{\texttt{"function"}} = \mathsf{None}$ *and* $d_{\texttt{"undefined"}} = \mathsf{None}$.

Symbol. *For $k =$ `"symbol"`, SKJS performs no refinement, so we omit f_k, g_k, d_k.*

Lemma 34 (`typeof` Refinement is Predicate-Sound (with Defaults))**.** *For each k in Definition 32, the transformer*

$$t \mapsto \mathsf{def}(t' \mapsto \mathsf{refine}^{f_k, g_k}(t'), d_k)(t)$$

is predicate-sound for $P_{\mathsf{typeof}=k}$. Moreover, the false-branch transformer $t \mapsto \mathsf{refine}^{\neg f_k, g_k}(t)$ is predicate-sound for $\neg P_{\mathsf{typeof}=k}$.

Proof. Fix k. Let $R_k(t) \triangleq \mathsf{refine}^{f_k,g_k}(t)$. We first show (f_k, g_k) is sound for $P_{\mathsf{typeof}=k}$ (Definition 18). Let $s \in \mathsf{SType}$ and $v \in [\![s]\!]$ with $v \in P_{\mathsf{typeof}=k}$. By definition of $P_{\mathsf{typeof}=k}$, $\mathsf{rtag}(v)$ matches the tag-set associated with k. By A0, $\mathsf{tag}(s) = \mathsf{rtag}(v)$, so $f_k(s) = \mathsf{true}$ (noting that for $k = \texttt{"object"}$, $\{2, 9, 10\}$ covers tag 9 in particular), and since g_k is the identity, $v \in [\![g_k(s)]\!]$. Thus (f_k, g_k) is sound and by Lemma 21, R_k is predicate-sound for $P_{\mathsf{typeof}=k}$.

For defaults, note that for each k with $d_k = \mathsf{Some}(s_k)$, we have $P_{\mathsf{typeof}=k} \subseteq [\![\mathsf{single}(s_k)]\!]$ by construction (e.g. string values inhabit `string`, and $\mathsf{ObjVal} \subseteq [\![\texttt{object}(F_\emptyset)]\!]$). By Lemma 22, $\mathsf{def}(R_k, d_k)$ is predicate-sound for $P_{\mathsf{typeof}=k}$.

For the false branch, the argument is analogous using $\neg f_k$ and $\neg P_{\mathsf{typeof}=k}$. □

5.5 Refinement Case Study: Algebraic Tag Equality

This corresponds to the implementation case where a discriminant field (written as `_tag` here) is compared to a string literal, and the checker refines algebraic types by filtering the set of possible constructor tags.

Definition 35 (Algebraic-Tag Predicate)**.** *Fix $\ell \in \mathsf{String}$ (a tag label) and define:*

$$P_{\mathsf{algTag}=\ell} \triangleq \{v \in \mathsf{Val} \mid v \in \mathsf{AlgVal} \wedge \mathsf{atag}(v) = \ell\}.$$

Definition 36 (Algebraic Tag Filter/Map Pairs)**.** *Fix $\ell \in \mathsf{String}$. Let $f_\ell^{\mathsf{alg}}(s) = \mathsf{true}$ for all s (the implementation does not drop non-algebraic alternatives for this refinement). Define mappers:*

- $g_\ell^{\mathsf{alg,true}}(\texttt{algebraic}(T)) \triangleq \texttt{algebraic}(T \cap \{\ell\})$*, and* $g_\ell^{\mathsf{alg,true}}(s) \triangleq s$ *for all other s.*
- $g_\ell^{\mathsf{alg,false}}(\texttt{algebraic}(T)) \triangleq \texttt{algebraic}(T \setminus \{\ell\})$*, and* $g_\ell^{\mathsf{alg,false}}(s) \triangleq s$ *for all other s.*

Lemma 37 (Algebraic Tag Equality is Predicate-Sound)**.** *For every $\ell \in \mathsf{String}$:*

$\mathsf{refine}^{f_\ell^{\mathsf{alg}}, g_\ell^{\mathsf{alg,true}}}$ *is predicate-sound for* $P_{\mathsf{algTag}=\ell}$, $\quad$ $\mathsf{refine}^{f_\ell^{\mathsf{alg}}, g_\ell^{\mathsf{alg,false}}}$ *is predicate-sound for* $\neg P_{\mathsf{algTag}=\ell}$.

Proof. For the true branch, we show $(f_\ell^{\mathsf{alg}}, g_\ell^{\mathsf{alg,true}})$ is sound for $P_{\mathsf{algTag}=\ell}$ (Definition 18) and apply Lemma 21. Let $s \in \mathsf{SType}$ and $v \in [\![s]\!]$ with $v \in P_{\mathsf{algTag}=\ell}$. Then $v \in \mathsf{AlgVal}$, so $\mathsf{rtag}(v) = 10$. By A0, $\mathsf{tag}(s) = 10$, hence s must be of the form $\texttt{algebraic}(T)$ for some finite T. By Definition 8, $v \in [\![\texttt{algebraic}(T)]\!]$ implies $\mathsf{atag}(v) \in T$. Since $\mathsf{atag}(v) = \ell$, we have $\ell \in T$, hence $v \in [\![\texttt{algebraic}(T \cap \{\ell\})]\!] = [\![g_\ell^{\mathsf{alg,true}}(s)]\!]$. Also $f_\ell^{\mathsf{alg}}(s) = \mathsf{true}$ by definition.

For the false branch, we show $(f_\ell^{\mathsf{alg}}, g_\ell^{\mathsf{alg,false}})$ is sound for $\neg P_{\mathsf{algTag}=\ell}$. Let $s \in \mathsf{SType}$ and $v \in [\![s]\!]$ with $v \notin P_{\mathsf{algTag}=\ell}$. We have $f_\ell^{\mathsf{alg}}(s) = \mathsf{true}$ by definition. If $v \notin \mathsf{AlgVal}$, then $\mathsf{tag}(s) \neq 10$ by A0, so s is not an algebraic type and $g_\ell^{\mathsf{alg,false}}(s) = s$; hence $v \in [\![g_\ell^{\mathsf{alg,false}}(s)]\!]$. If $v \in \mathsf{AlgVal}$ but $\mathsf{atag}(v) \neq \ell$, then $\mathsf{tag}(s) = 10$ by A0, so $s = \texttt{algebraic}(T)$ for some finite T. By Definition 8, $\mathsf{atag}(v) \in T$. Since $\mathsf{atag}(v) \neq \ell$, we have $\mathsf{atag}(v) \in T \setminus \{\ell\}$, hence $v \in [\![\texttt{algebraic}(T \setminus \{\ell\})]\!] = [\![g_\ell^{\mathsf{alg,false}}(s)]\!]$. □

5.6 Refinement Case Study: `instanceof`

The implementation refines `instanceof` tests only on the true branch, by assigning the local variable a (possibly generic) instance type. This refinement updates the *instance* component of TType rather than filtering STypes.

Definition 38 (`instanceof` Semantics and Implementation-Modeled Refinement)**.** *Assume a set of runtime class names* Class *and a predicate family*

$$P_{\mathsf{inst}} : \mathsf{Class} \to \mathcal{P}(\mathsf{Val})$$

capturing the runtime truth of ***`instanceof`*** *for each class. We assume the type system's instance forms in* TType *include a corresponding instance type* $T_{\mathsf{inst}}(C)$ *(with implementation-chosen type arguments), and a distinguished array instance type* T_{array}*.*

Axiom (`instanceof` inclusion). *For each class* C *we assume:*

$$v \in P_{\mathsf{inst}}(C) \implies v \in [\![T_{\mathsf{inst}}(C)]\!].$$

Array special case. *We assume* $P_{\mathsf{inst}}(\texttt{\textit{Array}}) \subseteq [\![T_{\mathsf{array}}]\!]$.

Lemma 39 (`instanceof`-True Overwrite is Predicate-Sound)**.** *For each class* C*, the constant transformer* $R_C(t) \triangleq T_{\mathsf{inst}}(C)$ *is predicate-sound for* $P_{\mathsf{inst}}(C)$*. Similarly, the constant transformer* $R_{\mathsf{array}}(t) \triangleq T_{\mathsf{array}}$ *is predicate-sound for* $P_{\mathsf{inst}}(\texttt{\textit{Array}})$*.*

Proof. Fix C and let $t \in \mathsf{TType}$. If $v \in [\![t]\!] \cap P_{\mathsf{inst}}(C)$, then $v \in P_{\mathsf{inst}}(C)$. By the axiom in Definition 38, $v \in [\![T_{\mathsf{inst}}(C)]\!] = [\![R_C(t)]\!]$. Thus $[\![t]\!] \cap P_{\mathsf{inst}}(C) \subseteq [\![R_C(t)]\!]$. The array case is identical using $P_{\mathsf{inst}}(\texttt{Array}) \subseteq [\![T_{\mathsf{array}}]\!]$. □

6 Type Difference (Diff)

Definition 40 (Diff Soundness Properties)**.** *Let* X *be a set equipped with a semantic interpretation* $[\![\cdot]\!] : X \to \mathcal{P}(\mathsf{Val})$*. An operation* $\mathsf{diff} : X \times X \to X$ *is* sound *(with respect to* $[\![\cdot]\!]$*) if for all* $x_1, x_2 \in X$*, letting* $x_3 = \mathsf{diff}(x_1, x_2)$*, the following properties hold:*

$$\textit{(P1)} \quad [\![x_3]\!] \subseteq [\![x_1]\!] \qquad \text{(under-approximation)}$$

$$\textit{(P2)} \quad [\![x_3]\!] \cup [\![x_2]\!] \supseteq [\![x_1]\!] \qquad \text{(coverage)}$$

In this document we instantiate X *with* STypes*,* Clause*,* Clauses*, and* TType*.*

Reader's guide. *These two properties imply that* $[\![x_3]\!]$ *is always a refinement of* $[\![x_1]\!]$*, and that it still covers all of* $[\![x_1]\!]$ *once* $[\![x_2]\!]$ *is added back. Equivalently (Lemma 41),* $[\![x_3]\!]$ *is an* over-approximation *of the true set difference* $[\![x_1]\!] \setminus [\![x_2]\!]$ *while staying within* $[\![x_1]\!]$*.*

Lemma 41 (Equivalent Characterization)**.** *For all* $x_1, x_2, x_3 \in X$*, (P1)* $\wedge$ *(P2) holds if and only if*

$$[\![x_1]\!] \setminus [\![x_2]\!] \subseteq [\![x_3]\!] \subseteq [\![x_1]\!].$$

Proof. ($\Rightarrow$) Let $v \in [\![x_1]\!] \setminus [\![x_2]\!]$. By (P2), $v \in [\![x_3]\!] \cup [\![x_2]\!]$. Since $v \notin [\![x_2]\!]$, we have $v \in [\![x_3]\!]$. Thus $[\![x_1]\!] \setminus [\![x_2]\!] \subseteq [\![x_3]\!]$. Combined with (P1): $[\![x_1]\!] \setminus [\![x_2]\!] \subseteq [\![x_3]\!] \subseteq [\![x_1]\!]$.

($\Leftarrow$) The right-hand inclusion $[\![x_3]\!] \subseteq [\![x_1]\!]$ is exactly (P1). For (P2),

$$[\![x_3]\!] \cup [\![x_2]\!] \supseteq ([\![x_1]\!] \setminus [\![x_2]\!]) \cup [\![x_2]\!] \supseteq [\![x_1]\!].$$

□

Definition 42 (Completeness)**.** *A diff operation is* complete *if:*

$$[\![\mathsf{diff}(t_1, t_2)]\!] = [\![t_1]\!] \setminus [\![t_2]\!]$$

By the equivalent characterization, (P1) $\wedge$ *(P2) implies* $[\![t_1]\!] \setminus [\![t_2]\!] \subseteq [\![\mathsf{diff}(t_1, t_2)]\!]$*. Completeness additionally requires the reverse:* $[\![\mathsf{diff}(t_1, t_2)]\!] \subseteq [\![t_1]\!] \setminus [\![t_2]\!]$*.*

6.1 STypes Diff

Definition 43 (STypes Diff)**.**

$$\mathsf{diff}_{\mathsf{STypes}}(L,R)(k) = \begin{cases} L(k) & \text{if } k \in \mathsf{tags}(L) \setminus \mathsf{tags}(R) \\ \textit{undefined} & \textit{otherwise} \end{cases}$$

Intuition. *Since an* STypes *value stores at most one representative per runtime tag, this diff only removes whole tags: it preserves exactly the tags present on the left but absent on the right.*

Lemma 44 (STypes Diff Satisfies (P1))**.** $[\![\mathsf{diff}_{\mathsf{STypes}}(L,R)]\!] \subseteq [\![L]\!]$.

Proof. By definition, $\mathsf{tags}(\mathsf{diff}_{\mathsf{STypes}}(L,R)) = \mathsf{tags}(L) \setminus \mathsf{tags}(R) \subseteq \mathsf{tags}(L)$, and for all $k \in \mathsf{tags}(\mathsf{diff}_{\mathsf{STypes}}(L,R))$ we have $\mathsf{diff}_{\mathsf{STypes}}(L,R)(k) = L(k)$. Therefore:

$$[\![\mathsf{diff}_{\mathsf{STypes}}(L,R)]\!] = \bigcup_{k \in \mathsf{tags}(\mathsf{diff}_{\mathsf{STypes}}(L,R))} [\![L(k)]\!] \subseteq \bigcup_{k \in \mathsf{tags}(L)} [\![L(k)]\!] = [\![L]\!].$$

□

Definition 45 (Same-Tag Coverage Condition)**.** *For $L, R \in \mathsf{STypes}$, define:*

$$\mathsf{STC}(L,R) \iff \forall k \in \mathsf{tags}(L) \cap \mathsf{tags}(R).\ [\![L(k)]\!] \subseteq [\![R(k)]\!]$$

Lemma 46 (STypes Diff Satisfies (P2) under STC)**.** *If* $\mathsf{STC}(L,R)$ *holds, then* $[\![\mathsf{diff}_{\mathsf{STypes}}(L,R)]\!] \cup [\![R]\!] \supseteq [\![L]\!]$.

Proof. Let $x \in [\![L]\!]$. Then $x \in [\![L(k)]\!]$ for some $k \in \mathsf{tags}(L)$.

- Case $k \notin \mathsf{tags}(R)$: Then $k \in \mathsf{tags}(\mathsf{diff}_{\mathsf{STypes}}(L,R))$ and $\mathsf{diff}_{\mathsf{STypes}}(L,R)(k) = L(k)$, so $x \in [\![\mathsf{diff}_{\mathsf{STypes}}(L,R)]\!]$.
- Case $k \in \mathsf{tags}(R)$: By STC, $[\![L(k)]\!] \subseteq [\![R(k)]\!] \subseteq [\![R]\!]$, so $x \in [\![R]\!]$.

□

Corollary 47 (STypes Diff under STC)**.** *If* $\mathsf{STC}(L,R)$ *holds, then (P2) holds. In particular, if* $L(k) = R(k)$ *for all* $k \in \mathsf{tags}(L) \cap \mathsf{tags}(R)$*, then (P2) holds.*

Lemma 48 (STypes Diff Completeness)**.** $\mathsf{diff}_{\mathsf{STypes}}$ *is complete (i.e.,* $[\![\mathsf{diff}_{\mathsf{STypes}}(L,R)]\!] = [\![L]\!] \setminus [\![R]\!]$*) if and only if* $\mathsf{STC}(L,R)$*.*

Proof. ($\Rightarrow$) Suppose completeness holds and let $k \in \mathsf{tags}(L) \cap \mathsf{tags}(R)$. Since $k \notin \mathsf{tags}(\mathsf{diff}_{\mathsf{STypes}}(L,R))$, the component $L(k)$ contributes no value to $[\![L]\!] \setminus [\![R]\!]$ under completeness, hence $[\![L(k)]\!] \subseteq [\![R]\!]$. By A1 and the "at most one entry per tag" invariant for STypes, $[\![L(k)]\!]$ can only overlap with $[\![R(k)]\!]$, so $[\![L(k)]\!] \subseteq [\![R(k)]\!]$.

($\Leftarrow$) We show $[\![\mathsf{diff}_{\mathsf{STypes}}(L,R)]\!] = [\![L]\!] \setminus [\![R]\!]$.

($\subseteq$) Let $x \in [\![\mathsf{diff}_{\mathsf{STypes}}(L,R)]\!]$. Then $x \in [\![L(k)]\!]$ for some $k \in \mathsf{tags}(L) \setminus \mathsf{tags}(R)$. By A1, $x \notin [\![R]\!]$ (no type in R has tag k), so $x \in [\![L]\!] \setminus [\![R]\!]$.

($\supseteq$) Let $x \in [\![L]\!] \setminus [\![R]\!]$. Then $x \in [\![L(k)]\!]$ for some $k \in \mathsf{tags}(L)$ and $x \notin [\![R]\!]$. If $k \in \mathsf{tags}(R)$, then by hypothesis $[\![L(k)]\!] \subseteq [\![R(k)]\!] \subseteq [\![R]\!]$, contradicting $x \notin [\![R]\!]$. So $k \notin \mathsf{tags}(R)$, meaning $k \in \mathsf{tags}(\mathsf{diff}_{\mathsf{STypes}}(L,R))$ and $x \in [\![\mathsf{diff}_{\mathsf{STypes}}(L,R)]\!]$. □

Corollary 49 (When STypes Diff is Incomplete)**.** $\mathsf{diff}_{\mathsf{STypes}}(L,R)$ *is incomplete when there exists* $k \in \mathsf{tags}(L) \cap \mathsf{tags}(R)$ *with* $[\![L(k)]\!] \not\subseteq [\![R(k)]\!]$*. Example:* $\mathsf{diff}_{\mathsf{STypes}}(\{4 \mapsto \mathtt{number}\}, \{4 \mapsto \mathtt{1}\}) = \emptyset$*, but* $[\![\mathtt{number}]\!] \setminus [\![\mathtt{1}]\!] \neq \emptyset$*.*

6.2 TInsts Diff

Definition 50 (TInsts Diff)**.**

$$\mathsf{diff}_{\mathsf{TInsts}}(L, R) = L \setminus R = \{i \in L \mid i \notin R\}$$

Note. *In the implementation,* TInsts *is represented as a collection of instance AST nodes and* $\mathsf{diff}_{\mathsf{TInsts}}$ *removes elements based on* structural equality *of instances. In this paper, that corresponds exactly to the usual mathematical set difference on the set* TInst *(i.e. membership/equality is the syntactic identity of instances, not semantic overlap).*

Lemma 51 (TInsts Diff Satisfies (P1) and (P2))**.** $[\![\mathsf{diff}_{\mathsf{TInsts}}(L, R)]\!] \subseteq [\![L]\!]$ *and* $[\![\mathsf{diff}_{\mathsf{TInsts}}(L, R)]\!] \cup [\![R]\!] \supseteq [\![L]\!]$.

Proof. (P1): $\mathsf{diff}_{\mathsf{TInsts}}(L, R) \subseteq L$, so $[\![\mathsf{diff}_{\mathsf{TInsts}}(L, R)]\!] \subseteq [\![L]\!]$.
(P2): Let $x \in [\![L]\!]$. Then $x \in [\![i]\!]$ for some $i \in L$.

- If $i \in R$: Then $x \in [\![i]\!] \subseteq [\![R]\!]$.
- If $i \notin R$: Then $i \in \mathsf{diff}_{\mathsf{TInsts}}(L, R)$, so $x \in [\![\mathsf{diff}_{\mathsf{TInsts}}(L, R)]\!]$.

□

Lemma 52 (TInsts Diff Completeness)**.** $\mathsf{diff}_{\mathsf{TInsts}}$ *is complete if and only if for all* $i \in L \setminus R$ *and* $j \in R$*:* $[\![i]\!] \cap [\![j]\!] = \emptyset$.

Proof. ($\Rightarrow$) Suppose completeness and let $i \in L \setminus R$ and $j \in R$. If $x \in [\![i]\!] \cap [\![j]\!]$, then $i \in \mathsf{diff}_{\mathsf{TInsts}}(L, R)$, so $x \in [\![\mathsf{diff}_{\mathsf{TInsts}}(L, R)]\!] = [\![L]\!] \setminus [\![R]\!]$. But $x \in [\![j]\!] \subseteq [\![R]\!]$, contradiction.
($\Leftarrow$) We show $[\![\mathsf{diff}_{\mathsf{TInsts}}(L, R)]\!] = [\![L]\!] \setminus [\![R]\!]$.
($\subseteq$) Let $x \in [\![\mathsf{diff}_{\mathsf{TInsts}}(L, R)]\!]$. Then $x \in [\![i]\!]$ for some $i \in L \setminus R$. For any $j \in R$, $[\![i]\!] \cap [\![j]\!] = \emptyset$ by hypothesis. Thus $x \notin [\![R]\!]$, so $x \in [\![L]\!] \setminus [\![R]\!]$.
($\supseteq$) Let $x \in [\![L]\!] \setminus [\![R]\!]$. Then $x \in [\![i]\!]$ for some $i \in L$. If $i \in R$, then $x \in [\![R]\!]$, contradiction. So $i \in L \setminus R = \mathsf{diff}_{\mathsf{TInsts}}(L, R)$. □

6.3 Clause Diff

Definition 53 (Clause Diff)**.** *For clauses* $c_1 = (\mathsf{vars}_1, \mathsf{stypes}_1)$ *and* $c_2 = (\mathsf{vars}_2, \mathsf{stypes}_2)$*:*

$$\mathsf{diff}_{\mathsf{Clause}}(c_1, c_2) = \begin{cases} \bot & \text{if } \mathsf{stypes}_2 = \mathsf{None} \\ c_1 & \text{if } \mathsf{stypes}_1 = \mathsf{None} \\ (\mathsf{vars}_1, \mathsf{Some}(\mathsf{diff}_{\mathsf{STypes}}(S_1, S_2))) & \text{if } \mathsf{stypes}_1 = \mathsf{Some}(S_1), \mathsf{stypes}_2 = \mathsf{Some}(S_2) \end{cases}$$

where $\bot$ *represents the empty type (dropped from results) with* $[\![\bot]\!] = \emptyset$.

Lemma 54 (Clause Diff Satisfies (P1))**.** $[\![\mathsf{diff}_{\mathsf{Clause}}(c_1, c_2)]\!] \subseteq [\![c_1]\!]$.

Proof. Case analysis:

- $\mathsf{stypes}_2 = \mathsf{None}$: Result is $\bot$. $[\![\bot]\!] = \emptyset \subseteq [\![c_1]\!]$.
- $\mathsf{stypes}_1 = \mathsf{None}$: Result is c_1. $[\![c_1]\!] \subseteq [\![c_1]\!]$.
- Both Some: $[\![(\mathsf{vars}_1, \mathsf{Some}(\mathsf{diff}_{\mathsf{STypes}}(S_1, S_2)))]\!] = \left(\bigcap_{v \in \mathsf{vars}_1} [\![v]\!]\right) \cap [\![\mathsf{diff}_{\mathsf{STypes}}(S_1, S_2)]\!] \subseteq \left(\bigcap_{v \in \mathsf{vars}_1} [\![v]\!]\right) \cap [\![S_1]\!] = [\![c_1]\!]$.

□

Lemma 55 (Clause Diff Satisfies (P2) Conditionally)**.** $\mathsf{diff}_{\mathsf{Clause}}$ *satisfies (P2) when:*

- *Case* $\mathsf{stypes}_2 = \mathsf{None}$*: Always for well-formed clauses (since then* c_2 *is the unique top clause and* $[\![c_2]\!] = \mathsf{Val} \supseteq [\![c_1]\!]$*).*
- *Case* $\mathsf{stypes}_1 = \mathsf{None}$*: Always (since result is* c_1*).*
- *Case both* Some*: When* $\mathsf{diff}_{\mathsf{STypes}}(S_1, S_2)$ *satisfies (P2).*

6.4 Clauses Diff

Definition 56 (Clauses Diff)**.** *For* $C_1, C_2 \in \mathsf{Clauses}$*, let* $c_\emptyset = C_2(\emptyset)$ *if* $\emptyset \in \mathrm{dom}(C_2)$*, else undefined.*

$$\mathsf{diff}_{\mathsf{Clauses}}(C_1, C_2)(k) = \begin{cases} \mathsf{diff}_{\mathsf{Clause}}(\mathsf{diff}_{\mathsf{Clause}}(C_1(k), C_2(k)), c_\emptyset) & \text{if } k \in \mathrm{dom}(C_2) \wedge c_\emptyset \text{ defined} \\ \mathsf{diff}_{\mathsf{Clause}}(C_1(k), C_2(k)) & \text{if } k \in \mathrm{dom}(C_2) \wedge c_\emptyset \text{ undefined} \\ \mathsf{diff}_{\mathsf{Clause}}(C_1(k), c_\emptyset) & \text{if } k \notin \mathrm{dom}(C_2) \wedge c_\emptyset \text{ defined} \\ C_1(k) & \text{if } k \notin \mathrm{dom}(C_2) \wedge c_\emptyset \text{ undefined} \end{cases}$$

for $k \in \mathrm{dom}(C_1)$*. Clauses mapped to* $\bot$ *are dropped.*

Note. *Whenever* $k \in \mathrm{dom}(C_2)$*, the call* $\mathsf{diff}_{\mathsf{Clause}}(C_1(k), C_2(k))$ *compares clauses stored under the* same key k*, so their variable-sets match by the* Clauses *well-formedness condition (keys agree with clause variables). The only subtraction that may use a different key is via* $c_\emptyset = C_2(\emptyset)$*, which is deliberately applied to every* $C_1(k)$*; this corresponds to the implementation behavior and explains why the right-hand clause's variables are not used by* $\mathsf{diff}_{\mathsf{Clause}}$*.*

Why the $\emptyset$-clause matters. *The distinguished key* $\emptyset$ *corresponds to clauses that do not require any type parameters. When present on the right, subtracting* $c_\emptyset$ *from every left clause models a "global" elimination that should apply regardless of which type-parameter constraints produced the left clause.*

Lemma 57 (Clauses Diff Satisfies (P1))**.** $[\![\mathsf{diff}_{\mathsf{Clauses}}(C_1, C_2)]\!] \subseteq [\![C_1]\!]$.

Proof. Each output clause is derived from $C_1(k)$ via $\mathsf{diff}_{\mathsf{Clause}}$, which satisfies (P1). □

6.5 TType Diff

Definition 58 (TType Diff)**.** *For* $t_1 = (\mathsf{insts}_1, \mathsf{clauses}_1)$ *and* $t_2 = (\mathsf{insts}_2, \mathsf{clauses}_2)$*:*

$$\mathsf{diff}_{\mathsf{TType}}(t_1, t_2) = (\mathsf{diff}_{\mathsf{TInsts}}(\mathsf{insts}_1, \mathsf{insts}_2), \mathsf{diff}_{\mathsf{Clauses}}(\mathsf{clauses}_1, \mathsf{clauses}_2))$$

Theorem 59 (TType Diff Satisfies (P1))**.** $[\![\mathsf{diff}_{\mathsf{TType}}(t_1, t_2)]\!] \subseteq [\![t_1]\!]$.

Proof.

$$\begin{aligned} [\![\mathsf{diff}_{\mathsf{TType}}(t_1, t_2)]\!] &= [\![\mathsf{diff}_{\mathsf{TInsts}}(\mathsf{insts}_1, \mathsf{insts}_2)]\!] \cup [\![\mathsf{diff}_{\mathsf{Clauses}}(\mathsf{clauses}_1, \mathsf{clauses}_2)]\!] \\ &\subseteq [\![\mathsf{insts}_1]\!] \cup [\![\mathsf{clauses}_1]\!] = [\![t_1]\!] \end{aligned}$$

by Lemmas 51 and 57. □

Theorem 60 (TType Diff Satisfies (P2) Conditionally)**.** $\mathsf{diff}_{\mathsf{TType}}$ *satisfies (P2) when all underlying* $\mathsf{diff}_{\mathsf{STypes}}$ *calls satisfy (P2). In particular, (P2) holds when* STC *holds for all same-tag eliminations.*

Theorem 61 (TType Diff Completeness)**.** $\mathsf{diff}_{\mathsf{TType}}$ *is complete when:*

1. $\mathsf{diff}_{\mathsf{TInsts}}$ *is complete (structurally distinct instances are semantically disjoint), and*
2. $\mathsf{diff}_{\mathsf{STypes}}$ *is complete for all clause pairs (same-tag implies* $[\![s]\!] \subseteq [\![s']\!]$*).*

Corollary 62 (Explicit Completeness Conditions for $\mathsf{diff}_{\mathsf{TType}}$)**.** *Let* $t_1 = (\mathsf{insts}_1, \mathsf{clauses}_1)$ *and* $t_2 = (\mathsf{insts}_2, \mathsf{clauses}_2)$*. Then* $\mathsf{diff}_{\mathsf{TType}}(t_1, t_2)$ *is complete (i.e.* $[\![\mathsf{diff}_{\mathsf{TType}}(t_1, t_2)]\!] = [\![t_1]\!] \setminus [\![t_2]\!]$*) under the following explicit hypotheses:*

1. ***Instance disjointness across sides:*** *for all* $i \in \mathsf{insts}_1$ *and* $j \in \mathsf{insts}_2$ *with* $i \neq j$*, we have* $[\![i]\!] \cap [\![j]\!] = \emptyset$*.*
2. ***Top-clause alignment:*** *if* $\mathsf{clauses}_1$ *is top (i.e. it contains the unique top clause* $(\emptyset, \mathsf{None})$*), then* $\mathsf{clauses}_2$ *is also top.*
3. ***Same-tag coverage for every clause subtraction:***
   (a) *For every* $k \in \mathrm{dom}(\mathsf{clauses}_1) \cap \mathrm{dom}(\mathsf{clauses}_2)$*, if* $\mathsf{clauses}_1(k) = (k, \mathsf{Some}(S_1))$ *and* $\mathsf{clauses}_2(k) = (k, \mathsf{Some}(S_2))$*, then* $\mathsf{STC}(S_1, S_2)$*.*
   (b) *If* $\emptyset \in \mathrm{dom}(\mathsf{clauses}_2)$ *and* $\mathsf{clauses}_2(\emptyset) = (\emptyset, \mathsf{Some}(S_0))$*, then for every* $k \in \mathrm{dom}(\mathsf{clauses}_1)$ *with* $\mathsf{clauses}_1(k) = (k, \mathsf{Some}(S_1))$*, we have* $\mathsf{STC}(S_1, S_0)$*.*

7 Application to Type Inference

Definition 63 (Subtyping Check with Validation)**.** *When checking* $u_1 \leq u_2$ *where* u_2 *contains type variable* T*:*

1. *Compute* $\mathsf{diff}(u_1, u_2)$ *and grow* T*'s lower bound.*
2. *Expand* u_1, u_2 *using the substitution.*
3. *Validate via* $\mathsf{isSub}(\mathit{expand}(u_1), \mathit{expand}(u_2))$*.*

Theorem 64 (Inference Soundness)**.** *If* isSub *correctly validates subtyping on expanded types, then type inference is sound regardless of whether* diff *satisfies (P2).*

Proof. The diff result is used only to guide inference (e.g. by accumulating bounds or hints about T). The acceptance/rejection decision is made solely by isSub on expanded types. Therefore, an imprecise diff can at worst cause incompleteness (e.g. by failing to discover a useful bound, leading to a later rejection), but it cannot cause an unsound acceptance when isSub is correct. □

8 Examples of Incompleteness

We present two concrete examples where diff is incomplete, i.e., $[\![\mathsf{diff}(t_1, t_2)]\!] \neq [\![t_1]\!] \setminus [\![t_2]\!]$ (it may be a strict subset or a strict superset of the true set difference).

8.1 Example 1: Same-Tag Literal Types

TypeScript code:

```
function f(x: 1 | 2): 2 {
  if (x === 1) { return 2; }
  return x;  // x should be narrowed to 2
}
```

Mathematical setup: Both `1` and `2` have the same runtime tag 4. Because STypes stores at most one SType per tag, it cannot precisely represent unions of same-tag literals such as `1 | 2`. In practice, the implementation merges same-tag literals into a single representative SType for the tag (often a wider type such as `number`), which loses the ability to subtract a single literal while keeping another.

Incompleteness: Since $\mathsf{tag}(\texttt{1}) = \mathsf{tag}(\texttt{2}) = 4$ and STypes stores at most one entry per tag, $\mathsf{diff}_{\mathsf{STypes}}$ cannot distinguish between literals of the same tag. For a simpler illustration:

$$\mathsf{diff}_{\mathsf{STypes}}(\{4 \mapsto \texttt{number}\}, \{4 \mapsto \texttt{1}\}) = \emptyset$$

But $[\![\texttt{number}]\!] \setminus [\![\texttt{1}]\!] =$ all numbers except $1 \neq \emptyset$.

The condition $\mathsf{STC}(L, R)$ fails: $[\![\texttt{number}]\!] \not\subseteq [\![\texttt{1}]\!]$.

8.2 Example 2: Top Type (unknown)

TypeScript code (requires `--strictNullChecks`):

```
function f(x: unknown): {} {
  if (x !== null && x !== undefined) {
    return x;  // x should be narrowed to {}
  }
  return {};
}
```

Mathematical setup: The type `unknown` is represented as a clause with $\mathsf{stypes} = \mathsf{None}$ (top). Let $c_1 = (\emptyset, \mathsf{None})$ represent `unknown`. Let c_2 represent `null | undefined`: $c_2 = (\emptyset, \mathsf{Some}(\{0 \mapsto \texttt{undefined}, 2 \mapsto \texttt{null}\}))$.

Incompleteness: By definition of $\mathsf{diff}_{\mathsf{Clause}}$:

$$\mathsf{diff}_{\mathsf{Clause}}(c_1, c_2) = c_1 \quad (\text{since } \mathsf{stypes}_1 = \mathsf{None})$$

So $[\![\mathsf{diff}_{\mathsf{Clause}}(c_1, c_2)]\!] = [\![c_1]\!] = \mathsf{Val}$ (all values).

But the true set difference is:

$$[\![c_1]\!] \setminus [\![c_2]\!] = \mathsf{Val} \setminus ([\![\texttt{null}]\!] \cup [\![\texttt{undefined}]\!]) \neq \mathsf{Val}$$

The result is incomplete: diff returns `unknown` instead of the narrowed type {} (non-nullish values). This is because the clause diff cannot "subtract from top"—it simply returns the left clause unchanged when $\mathsf{stypes}_1 = \mathsf{None}$.